

Introduction to Algorithms: 6.006

[recitation] Recitation 15: Dynamic Programming (动态规划)

Massachusetts Institute of Technology Instructors: Erik Demaine, Jason Ku, and Justin Solomon

MIT 6.006 Spring 2020, Recitation 15 讲义 (源 PDF:
6022dc36557c5e5df2ad0a77e9463855_MIT6_006S20_r15.pdf, 6 页)

[section] Dynamic Programming Overview (动态规划总览)

本讲义讨论的 dynamic programming (动态规划) 将 divide and conquer (分治) 型递推 generalize (推广) 到子问题依赖关系构成 directed acyclic graph (有向无环图, 简称 DAG) 而非 tree (树) 的情形。DP 常用于 optimization problems (最优化问题) ——你要最大化或最小化单个标量值; 或 counting problems (计数问题) ——你要数出所有可能性。用动态规划解题时, 我们遵循以下步骤, 它们构成一个 recursive problem solving framework (递归解题框架)。

[step] How to Solve a Problem Recursively (SRT BOT) (如何递归地解一个问题)

1. Subproblem definition (子问题定义) ——子问题 $x \in X$:

- 用参数在 words (自然语言) 层面描述子问题的含义
- 通常取输入的 subsets (子集): prefixes (前缀)、suffixes (后缀)、contiguous subsequences (连续子序列)
- 常记录 partial state (部分状态): 通过递增某些 auxiliary variables (辅助变量) 来增加子问题

2. Relate subproblem solutions recursively (递归关联子问题解)

$$x(i) = f(x(j), \dots) \quad \text{对某个或多个 } j < i$$

3. Topological order (拓扑序) ——论证该关联是无环的, 子问题构成一个 DAG。

4. Base cases (基例)

- 对所有 (可达的) 关联不再适用/不再起作用的 independent subproblems (独立子问题) 给出解

5. Original problem (原问题)

- 说明如何从子问题的解计算出原问题的解
- 可能需要使用 parent pointers (父指针) 恢复 actual solution (实际方案), 而不仅是 objective function (目标函数) 值

6. Time analysis (时间分析)

$\sum_{x \in X} \text{work}(x)$, 若对所有 $x \in X$ 有 $\text{work}(x) = O(W)$, 则为 $|X| \cdot O(W)$

- $\text{work}(x)$ 度量关联式中的非递归工作量; 把递归调用视为只花 $O(1)$ 时间

[note] Note (译者注): SRT BOT 是 Demaine 为记忆六步法编的 acronym (首字母缩略词), 按他的原话, 这个模板就是「递归算法所需全部步骤的清单」, 而 DP 只是在它之上叠加一个新想法——memoization (记忆化), 即复用之前做过的计算。🧠 Demaine 在 Lecture 15 开场解释词源: "Instead of memorization, it's memoization, because we're going to write things down in a memo pad." (翻译: 不是 memorization (记忆), 而是 memoization (记忆化), 因为我们要把东西写进一个备忘录便签本里。) ——"memo" (备忘录) 这个核心意象提醒你: DP 的本质是把递归树的重复计算缓存起来。这与你在大一 CSAPP 学过的 caching (缓存) 思想同构: DP 是"时间维度上的缓存"——同样的子问题结果不重复算。六步里最容易被初学者跳过的其实是第 3 步拓扑序论证: 它保证递归能终止 (无环), 相当于证明"这份递推定义良好"。

[note] Note (译者注): SRT BOT 的第 3 步拓扑序论证, 对你来说是老朋友——你在 6.006 早先学的 DAG 松弛算法 (先拓扑排序、再按序松弛出边) 正是同一论证的另一副面孔。当时你掌握的论证是: "按拓扑序处理顶点时, 该顶点的所有入边信息已经齐备, 此刻松弛得到的就是最终距离"; SRT BOT 第 3 步要求你证明"子问题依赖构成 DAG", 理由完全同构——按拓扑序处理子问题时, 它依赖的所有更小子问题都已算好, 递推式才有意义。我们把这种"先证无环、再证按序处理时信息齐备"的策略命名为「拓扑序复用论证」, 它在 DP 里的变体更省事: 子问题下标严格递增/递减, 拓扑序就不言自明 (本讲 Blackjack 的"只依赖更大的 i "、Text Justification 的"只依赖更小的 i "、区间和的"依赖更小的 $j - i$ "都是如此), 根本不必跑 Full-DFS 找序。🧠 Demaine 在 Lecture 15 把这层等价直接挑明: "DAG shortest path is itself a DP." (翻译: DAG 最短路径本身就是动态规划。) 而你会在 Lecture 16 的 LCS/LIS 里看到这句话的反面: 把 DP 画成一张图、在上面跑 DAG 最短路径。图算法与 DP 的唯一差别只是视角——图里拓扑序是"算出来的", DP 里拓扑序是"设计出来的"。

[section] Implementation (实现)

一旦选定了子问题并找到依赖关系的 DAG, 就有两种主要实现方法, 它们功能等价但实现方式不同:

- **Top down (自顶向下)** 从 roots (根, 即没有入边的顶点) 开始求值递归。每次递归调用结束时, 把算出的子问题解记录进 memo (备忘录); 每次递归调用开始时, 先查 memo 看该

子问题是否已经解过。

- **Bottom up (自底向上)** 按照子问题依赖 DAG 的一个 topological sort order (拓扑排序序) 依次计算每个子问题, 同样把每个子问题解记入 memo 供后续子问题使用。通常子问题会被设计成拓扑序显而易见——尤其是当子问题只依赖参数更小的子问题时, 为此做一次 DFS (深度优先搜索) 找序往往没有必要。

实现层面分两种: top down (自顶向下) 是递归视角, bottom up (自底向上) 则把递归展开 (unroll)。两种实现都正确且常用。两种实现都用 memoization (记忆化) 来记住之前子问题的计算结果。虽然典型做法是把所有求值过的子问题都记忆化, 但常常可以少记一些子问题——尤其是当子问题按“轮次” (rounds) 出现时。

我们往往不只是想要被最优化的那个值, 还想返回导致最优化值的那条子问题路径。要 reconstruct (重构) 答案, 除了被最优化的值, 还需要维护 auxiliary information (辅助信息)。在维护最优值的同时, 可以维护指向当前子问题解所依赖的那个 (或那些) 子问题的 parent pointers (父指针)。这与 shortest path (最短路径) 问题中维护父指针是类似的。

[note] Note (译者注) : 这一段两个要点与 Lecture 15 的讲解直接呼应。第一, top down 与 bottom up 的功能等价性: 🐼 Demaine 的说法是“every DP algorithm is going to have that structure... using recursion and memoization together” (翻译: 每个 DP 算法都会有那个结构……把递归和记忆化一起使用。)——top down 是“递归 + 查表”, bottom up 是“按拓扑序填表”, 两者算的是同一张 memo 表。第二, “父子针”不是 DP 的发明, 而是你在 6.006 早先学的 shortest path (最短路径) 算法的老工具: Dijkstra 里每个顶点记录 predecessor (前驱) 用于重构路径, DP 里每个子问题记录“最优解依赖哪个更小的子问题”用于重构方案。重构的代价是 $O(1)$ 额外空间/子问题, 但能让你从“只知道最优值”升级到“给出具体方案”——这是面试与工程中区分“会 DP”和“真会 DP”的分水岭。当你只想求值 (如只要最大赢局数) 时可以省掉父指针, 但一旦要求输出方案 (如重构最优的击球顺序、断行位置), 父指针就是必需品。

[exercise] Exercise: Simplified Blackjack (简化版二十一点)

我们定义一名 player (玩家) 与一名 dealer (庄家) 之间简化版 blackjack (二十一点) 游戏。一副牌是有序的 n 张牌序列 $D = (c_1, \dots, c_n)$, 每张牌 c_i 是 1 到 10 之间的整数 (与真实二十一点不同, 这里的 ace (A 牌) 恒为 1)。游戏按 round (轮) 进行。一轮中: 庄家先抽牌堆最上面两张 (最初是 c_1, c_2), 然后玩家抽接下来两张 (最初是 c_3, c_4), 接着玩家可以选择再抽或不抽一张额外牌 (称为 hit, 要牌)。

若玩家手牌的点数 (即本轮玩家所抽牌点数之和) ≤ 21 且超过庄家手牌点数, 则玩家赢得该轮; 否则玩家输掉该轮。当某一轮结束时牌堆剩余少于 5 张牌, 游戏结束。

给定一个顺序已知的 n 张牌牌堆, 描述一个 $O(n)$ 时间算法, 确定玩家用这副牌玩简化版二十一点最多能赢多少轮。

[note] Note (译者注) : 这个 Blackjack 题与 Lecture 15 的 Bowling (保龄球) 题是同一设计模式的两张面孔：都是"从左到右消费输入、每步有少量离散选择"的 optimization problem。Bowling 里你每次可以选择击 1 个瓶或 2 个瓶；Blackjack 里每轮你可以选择 hit (用 5 张牌) 或不 hit (用 4 张牌)。🐼 Demaine 在 Lecture 15 介绍这类题时用的方法论是："you try to ask a question whose answer would let you reduce to a smaller subproblem" (翻译：你试着提出一个问题，这个问题的答案会帮你把问题归约到更小的子问题。) ——Blackjack 里那个"问题"就是：本轮玩家要不要 hit？答案只有两种，各自对应唯一的消费张数，于是归约到 $x(i+4)$ 或 $x(i+5)$ 。**设计 DP 的第一性原理：找到那个"每步必须做出的局部决策" (guess)，对它的所有可能取值 locally brute-force (局部暴力枚举)，其余交给递归。**这正是"careful brute force (精细的暴力枚举)"的由来。

[note] Note (译者注) : guess 这个动作，你在学 Bellman-Ford 时其实已经亲手做过一次。回想你当时的质疑：Bellman-Ford 要盲目地扫描所有边 $V-1$ 轮，太笨拙——于是你提出只把"距离刚被更新的顶点"放进队列、每次只松弛它的出边，这正是 SPFA (Shortest Path Faster Algorithm) 的核心思想；顺带一提，SPFA 在官方维基百科有独立词条，你当年独立"发明"的算法是有正式学术身份的。你当时用的思维链条与这里的 guess 是同一个「**局部决策 + 其余交给递归/松弛**」第一性原理：你不必一次想清楚整条最短路径，只需要回答"这一轮哪个顶点的距离可能变了、要不要松弛它的邻居"，剩下交给队列循环反复传播；同理，Blackjack 里你不必一次规划完整局，只需要回答"这一轮 hit 还是不 hit"，剩下交给 $x(i+4)$ 或 $x(i+5)$ 。🐼 Demaine 在 Lecture 15 用 Bowling 示范了这句心法："The big idea here is to just think about... what could I do to pin i?" (翻译：这里的大想法就是只考虑.....我能对第 i 个球瓶做什么？) ——三件事同构：SPFA 的 guess 是"被更新的顶点"，Bowling 的 guess 是"第一个球瓶的三种命运"，Blackjack 的 guess 是"玩家要牌与否"；guess 规模因题而异，方法论同一：找到每步必须做的局部决策、对其所有取值局部暴力枚举、其余交给递归去传播。你当初独立"发明"SPFA 的体验，就是一次未经命名的 guess 实践——现在它有了正式名字。

Solution (解答):

1. Subproblems (子问题)

- 选择 suffixes (后缀)
- $x(i)$: 用牌 $(c_i, \dots, c_n)$ 玩 blackjack 玩家能赢的最大轮数

2. Relate (关联)

- Guess (猜测) 玩家是否要牌 (hit)
- 庄家手牌点数恒为 $c_i + c_{i+1}$
- 玩家手牌点数要么是：
 - $c_{i+2} + c_{i+3}$ (不 hit, 本轮用 4 张牌), 或
 - $c_{i+2} + c_{i+3} + c_{i+4}$ (hit, 本轮用 5 张牌)
- 令 $w(d, p)$ 为给定手牌点数 d 与 p (庄家与玩家) 时的本轮结果
 - 玩家胜: $w(d, p) = 1$ 当 $d < p \leq 21$
 - 玩家负: $w(d, p) = 0$ 否则 (当 $p \leq d$ 或 $21 < p$)
- $x(i) = \max \{w(c_i + c_{i+1}, c_{i+2} + c_{i+3}) + x(i+4), w(c_i + c_{i+1}, c_{i+2} + c_{i+3} + c_{i+4}) + x(i+5)\}$
- (当 $n - (i - 1) \geq 5$, 即 $i \leq n - 4$ 时)

3. Topo (拓扑序)

- 子问题 $x(i)$ 只依赖严格更大的 i , 因此无环。

4. Base (基例)

- $x(n - 3) = x(n - 2) = x(n - 1) = x(n) = x(n + 1) = 0$
- (剩余牌数不足以再开一轮)

5. Original (原问题)

- 用递归 top down 或迭代 bottom up 求解 $i \in \{1, \dots, n + 1\}$ 的所有 $x(i)$
- $x(1)$: 用整副牌玩 blackjack 玩家能赢的最大轮数

6. Time (时间分析)

- 子问题个数: $n + 1$
- 每个子问题工作量: $O(1)$
- 总运行时间: $O(n)$

[note] Note (译者注): 两个值得细看的细节。其一, 下标边界 $i \leq n - 4$ 的来历: 一轮至少要消耗 4 张牌 (庄家 2 + 玩家 2), 要保证"能开一轮"必须满足从 c_i 起至少有 5 张牌吗? 不是——若剩下恰好 4 张牌, 本轮可以开 (庄家抽 2、玩家抽 2), 但若玩家选择 hit 就会牌不够, 所以公式里 hit 分支只在牌足够时合法; 讲义用 $n - (i - 1) \geq 5 \iff i \leq n - 4$ 作为递推式成立的域, 而 $x(n - 3) = \dots = x(n + 1) = 0$ 这些基例覆盖了"剩余不足 5 张"的情形 ($x(n - 3)$ 剩 4 张: 可以开最后一轮不 hit, 也可以直接记 0——由于 $x(n - 3)$ 之后没有更小的合法递推, 它本身就是终止点)。其二, 参数计数: 子问题空间是 $O(n)$ 个后缀, 每个后缀只有 $O(1)$ 种转移, 所以总复杂度是 $n \cdot O(1) = O(n)$ 。这个"子问题数 $\times$ 每子问题工作量"的乘法结构就是 SRT BOT 第 6 步公式 $\sum_x \text{work}(x)$ 的直接应用, 也是判断一个 DP 设计是否高效的第一指标。

[note] Note (译者注) : 这道题第 1 步"选择 suffixes 作为子问题", 与你 Problem Session 7 学过的状态图复制 (state graph copying) 是同一个建模思想的两副躯体。回想你当时的 Critter Collection (Problem 7-4): 背包容量 k 是约束, 解法是把每个 clearing 复制成 $k + 1$ 个顶点 $v_{c,i}$, 代表"人在 clearing c 、包里还有 i 个空球"——背包容量被展开成图的"层", 带状态限制的寻路就此变成新图上一次标准最短路径查询。Blackjack 里发生的事一模一样: 你给输入加了一个下标维度 i , $x(i)$ 表示"从第 i 张牌开始玩", 这个 i 就是把"已用掉多少张牌"这个状态展开成一维。把两者并置, 得到统一的建模视角, 我们称之为「**状态维度展开**」: 遇到带约束的问题, 把状态作为新维度展开、把约束吸收进状态空间, 剩下的交给已知算法。区别在于: 状态图复制构造显式的图, 允许有环, 答案是一次查询; DP 构造隐式的子问题 DAG, 必须无环, 答案是按拓扑序填表。你在 Prob7-4 用的"带状态限制的寻路"思考路径, 在 DP 里就是"带状态定义的子问题"——两者都是先把问题升维, 再在更高维空间求解。

[exercise] Exercise: Text Justification (文本对齐)

Text Justification (文本对齐) 是这样一类问题: 把 n 个以空格分隔的单词序列排进一个固定宽度为 s 的列中, 使得单词之间的 white-space (空白) 总量最小。每个单词可用其宽度 $w_i < s$ 表示。最小化一行内空白的一个好办法是 minimize (最小化) 该行的 badness (不良度)。假设一行包含从 w_i 到 w_j 的单词, 该行 badness 定义为:

$$b(i, j) = (s - (w_i + \dots + w_j))^3 \quad \text{若 } s > (w_i + \dots + w_j), \quad b(i, j) = \infty \text{ 否则}$$

一个好的文本对齐算法会把单词划分成行, 使得所有含单词的行的 badness 总和最小。三次方 (cubic power) 会重度惩罚一行内的大片空白。Microsoft Word 用贪心算法对齐文本: 在换行前尽量往一行里塞尽可能多的单词。这个算法可能产生一些真的很差的行。LATEX 则相反, 用动态规划格式化文本以最小化这个空白度量。描述一个 $O(n^2)$ 算法, 把 n 个单词排进宽度为 s 的列中, 使所有行的 badness 总和最小。

[note] Note (译者注) : 这个练习是"贪心失败、DP 胜出"的经典教学案例, 也解释了为什么你每天用的工具背后藏着 DP: Microsoft Word 的 greedy (贪心) 断行——每行尽量塞满——在长单词先行的场景下会把大空白挤到行尾, 产生视觉上的"河流"; 而 Donald Knuth 在 1970 年代末为 TEX (现代 LATEX 的排版引擎) 实现的行断算法就是动态规划, 他 1981 年与 Plass 合著的论文 *Breaking Paragraphs into Lines* 正是此题的工业级版本, 后来被 Adobe 的 InDesign 等排版软件采用。**为什么三次方?** 线性或平方惩罚下, 把一行空白摊薄成多行空白可能不被惩罚, 而立方函数让"一行内 k 个单位空白"的成本 (k^3) 远大于"拆成 k 个单行各 1 个单位空白"的成本 (k), 从而逼出"行间空白均匀分布"的视觉目标——这是惩罚函数设计服务于目标函数的第一性原理例子。

Solution (解答):

1. Subproblems (子问题)

- 选择 suffixes (后缀) 作为子问题
- $x(i)$: 把单词 w_i 到 w_{n-1} 格式化后的最小 badness 总和

2. Relate (关联)

- 第一行必须在某个单词处断开, 所以尝试所有可能性
- $x(i) = \min \{b(i, j) + x(j + 1) \mid i \leq j < n\}$

3. Topo (拓扑序)

- 子问题 $x(i)$ 只依赖严格更大的 i , 因此无环。

4. Base (基例)

- $x(n) = 0$: 格式化零个单词的 badness 是零

5. Original (原问题)

- 用递归 top down 或迭代 bottom up 求解子问题
- 原问题的解是 $x(0)$
- 存储 parent pointers (父指针) 以重构断行位置

6. Time (时间分析)

- 子问题个数: $O(n)$
- 每个子问题工作量: $O(n^2)$
- 总运行时间: $O(n^3)$

[note] Note (译者注): 注意第 2 步把"第一行断在哪"做成 guess: 对每个起点 i , 尝试所有可能的断点 j ($O(n)$ 个), 每个断点的成本是 $b(i, j)$ 加剩余后缀的最优解。这里的 min (最小值) 对应你在 6.042J 学过的"最优子结构"直觉: 如果 $x(i)$ 是全局最优, 那么断点之后的 $x(j + 1)$ 也必须是子问题最优, 否则可以替换得到更优——这是 DP 正确性的论证根基 (counterfactual reasoning, 反证法)。而 $O(n^3)$ 来自三处相乘: $O(n)$ 个子问题 $\times$ 每个 $O(n)$ 个断点 $\times$ 每次计算 $b(i, j)$ 需要 $O(n)$ 次加法求和 $w_i + \dots + w_j$ 。下一步优化的切入点正是这最后一个 $O(n)$ ——这正是下一页"预计算"的动机。

[section] Optimization (优化)

- 计算 badness $b(i, j)$ 可能要花 linear time (线性时间) !
- 如果能用 $O(1)$ 时间预计算并记住每个 $b(i, j)$, 那么:
 - 每个子问题工作量: $O(n)$
 - 总运行时间: $O(n^2)$

用 $O(n^2)$ 时间预计算所有 $b(i, j)$ ，也使用动态规划！

1. Subproblems (子问题)

- $x(i, j)$: 单词 w_i 到 w_j 的长度之和

2. Relate (关联)

- $x(i, j) = \sum_k w_k$ ——计算要花 $O(j - i)$ 时间，慢！
- $x(i, j) = x(i, j - 1) + w_j$ ——计算只要 $O(1)$ 时间，快！

3. Topo (拓扑序)

- 子问题 $x(i, j)$ 只依赖严格更小的 $j - i$ (区间长度)，因此无环。

4. Base (基例)

- $x(i, i) = w_i$ ，对所有 $0 \leq i < n$ ，即只有一个单词

5. Original (原问题)

- 用递归 top down 或迭代 bottom up 求解子问题
- 用 $O(1)$ 时间计算每个 $b(i, j) = (s - x(i, j))^3$

6. Time (时间分析)

- 子问题个数: $O(n^2)$
- 每个子问题工作量: $O(1)$
- 总运行时间: $O(n^2)$

[note] Note (译者注) : 这个优化是 DP 里最优雅的"用 DP 加速 DP"技巧: 外部 DP 需要区间和 $w_i + \dots + w_j$ ，于是内部再用一个二维 DP 预计算所有区间和，递推 $x(i, j) = x(i, j - 1) + w_j$ 把每个区间和从 $O(j - i)$ 压到 $O(1)$ ——这正是你熟悉的 **prefix sum (前缀和)** 思想的二维版。你在 6.006 前面学过的 range sum query (区间和查询) 用的就是同一招: 先 $O(n)$ 预处理前缀和，任何区间和 $O(1)$ 查询。这里由于断行需要所有 $O(n^2)$ 个区间的和，前缀和数组 (一维) 其实已经够用: $x(i, j) = \text{prefix}[j + 1] - \text{prefix}[i]$ 。讲义写成二维 DP 是为了演示"同一范式可以递归套用"——内层 DP 有 $O(n^2)$ 个子问题、每个 $O(1)$ 转移，完全符合 SRT BOT 框架 (拓扑序是区间长度 $j - i$ 严格递增)。**总结这条优化链: $O(n^3) \rightarrow O(n^2)$** ，全部来自把重复的加法求和替换成记忆化查询——又一次"recurse but re-use (递归但复用)"。

[note] Note (译者注) : 这条优化链与你在大一《计算机科学数学》9.43(e) 学过的 LIS 优化是同一个母题，值得并置记忆。你当时学过 LIS 的两种解法： $O(n^2)$ 的 DP（每个位置扫描全部前驱找满足条件的最大 $x(j)$ ）与 $O(n \log n)$ 的贪心+二分（维护单调的 tails 数组、不断“降低门槛”——官方维基百科称其为 patience sorting，纸牌接龙排序）。Text Justification 的优化与之共享同一思维模式，我们称之为「重复计算审计」——先问每步在重复算什么。这里 $O(n^3)$ 的元凶是求和 $w_i + \dots + w_j$ ：每个 $b(i, j)$ 都要 $O(n)$ 次加法，而 $O(n^2)$ 个区间和之间大量重叠，于是用前缀和/二维 DP 全部预计算，把重复加法变成 $O(1)$ 查表；LIS 那边， $O(n^2)$ 里每个位置都在重复扫描全体前驱，tails 数组把“扫描全部”变成“二分定位一个”，同样是把重复劳动结构化。区别在于手段：Text Justification 是“把重复算过的东西存起来”（记忆化、复用），LIS 是“把重复要问的问题压进一个单调结构”（额外省出 $\log n$ 因子）；共同点是先审计出重复项，再决定用缓存还是用结构消灭它。用复杂度语言说： $O(n^3) \rightarrow O(n^2)$ 去掉的是一个“重复加法因子 n ”， $O(n^2) \rightarrow O(n \log n)$ 去掉的是一个“重复扫描因子 n ”——都是先看见重复，再消灭重复。

Recitation 15 结束。译者注 11 条：SRT BOT 框架与 memoization 词源、拓扑序复用论证（DAG 松弛同构）、实现与父指针、Blackjack 设计模式、guess 活教材（SPFA 同构）、Blackjack 边界与复杂度、状态维度展开（状态图复制同构）、Text Justification 与优化链、重复计算审计母题（LIS 并置）。

[reference] Reference (来源与延伸)

- MIT 6.006 Spring 2020, Recitation 15 (本讲义)
- Lecture 15: Dynamic Programming, Part 1 — SRTBOT, Fib, DAGs, Bowling (视频字幕已引用)
- CLRS 4th ed. §14 (Dynamic Programming)；Knuth & Plass 1981, *Breaking Paragraphs into Lines* (官方维基百科 Knuth–Plass line-breaking algorithm 词条，TeX 的 discrete DP 断行)
- 官方维基百科 (2026-06 离线版)：Dynamic programming (Bellman 1950s 提出)、Shortest Path Faster Algorithm、Longest increasing subsequence / Patience sorting
- 关联讲义：`lec16-dyn-prog-subproblems.bilingual.md` (LCS/LIS/Coins)、`prob7sol.bilingual.md` (状态图复制, Prob7-4)

MIT OpenCourseWare — <https://ocw.mit.edu> — 6.006 Introduction to Algorithms, Spring 2020.
Terms of Use: <https://ocw.mit.edu/terms>